

Subiect 04 – Aplicație pentru restaurant

Cerințe obligatorii

1. Pattern-urile implementate trebuie să respecte definiția din GoF discutată în cadrul cursurilor și laboratoarelor. Nu sunt acceptate variații sau implementări incomplete.
2. Pattern-ul trebuie implementat în totalitate corect pentru a fi luat în calcul.
3. Soluția nu conține erori de compilare.
4. Pattern-urile pot fi tratate distinct sau pot fi implementate pe același set de clase.
5. Implementările care nu au legătura funcțională cu cerințele din subiect NU vor fi luate în calcul (preluare unui exemplu din alte surse nu va fi punctată).
6. NU este permisă modificare claselor primite.
7. Soluțiile vor fi verificate încrucișat folosind MOSS. Nu este permisă partajarea de cod între studenți. Soluțiile care au un grad de similitudine mai mare de 30% vor fi anulate.

Cerințe Clean Code obligatorii (soluția este depunctată cu câte 2 puncte pentru fiecare cerință ce nu este respectată) - maxim se pot pierde 8 puncte

1. Pentru denumirea claselor, funcțiilor, testelor unitare, atributelor și a variabilelor se respecta convenția de nume de tip Java Mix CamelCase;
2. Pattern-urile și clasa ce conține metoda main() sunt definite în pachete distincte ce au forma *cts.num.prenume.gNrGrupa.denumire_pattern*, *cts.num.prenume.gNrGrupa.main* (studenții din anul suplimentar trec "as" în loc de gNrGrupa);
3. Clasele și metodele sunt implementate respectând principiile KISS, DRY și SOLID (atenție la DIP);
4. Denumirile de clase, metode, variabile, precum și mesajele afișate la consola trebuie să aibă legătura cu subiectul primit (nu sunt acceptate denumiri generice). Funcțional, metodele vor afișa mesaje la consola care să simuleze acțiunea cerută sau vor implementa prelucrări simple.

Se dezvoltă o aplicație software destinată unui restaurant

3p. Se dorește dezvoltarea unei aplicații prin care clienții pot plasa comenzi într-un restaurant. O comandă poate fi trimisă către bucătărie, anulată, repetată sau salvată pentru procesare ulterioară. Se dorește ca fiecare acțiune asupra comenzii să fie reprezentată printr-un obiect separat, astfel încât acțiunile să poată fi stocate, executate ulterior sau combinate în liste de comenzi. Pentru implementare se va folosi interfața *AbstractActiuneComanda*.

1.5p. Să se testeze soluția prin:

- crearea unor acțiuni pentru trimitere, anulare și repetare comandă;
- executarea acțiunilor printr-un obiect responsabil cu gestionarea lor;
- salvarea mai multor acțiuni într-o listă și executarea lor succesivă.

3p. În aplicația restaurantului, un produs de bază, precum pizza, burger sau salată, poate primi opțiuni suplimentare: extra brânză, sos special, topping premium, ambalaj eco sau livrare rapidă. Aceste opțiuni trebuie să poată fi combinate liber, fără crearea unei clase separate pentru fiecare combinație posibilă. Pentru implementare se va folosi interfața *AbstractProdusRestaurant*.

1.5p. Să se testeze soluția prin:

- crearea unui produs simplu;

- adăugarea mai multor opțiuni suplimentare;
- afișarea descrierii finale și a prețului total.

```
public interface AbstractActiuneComanda {  
    void executa();  
}
```

```
public interface AbstractProdusRestaurant {  
    String obtineDescriere();  
    double calculeazaPret();  
}
```